Week 8 Guide: From CSV Tables to Cloud Database Models

Why we are starting with CSVs

Before using a database service, it helps to see that a table is simply a structured way to store facts. A CSV file is not a full database system, but it is a clear way to see rows, columns, identifiers, and repeated values.

This week starts with three CSV files in `week_08/tables/`: - `users.csv` - `workspaces.csv` - `files.csv`

Together they describe a fictional cloud workspace platform used for class materials and advising records.

What a table means

A **table** stores one kind of thing. Each **row** is one record. Each **column** is one attribute about that record.

In this week's sample: - `users.csv` stores people - `workspaces.csv` stores cloud workspaces - `files.csv` stores files inside those workspaces

Two ideas matter immediately: - some columns uniquely identify a row, such as `user_id` or `workspace_id` - some columns point to a row in another table, such as `owner_user_id` or `workspace_id`

That second idea is how tables become related instead of isolated.

The three source tables

`users.csv`

Each row is one person.

user_id	full_name	role	department	preferred_region
U1001	Ana Rivera	student	Data Science	us-east-1
U1003	Priya Shah	teaching_assistant	Computer Systems	us-east-2

`workspaces.csv`

Each row is one cloud workspace. `owner_user_id` points back to `users.csv`.

workspace_id	workspace_name	owner_user_id	data_classification	cloud_provider
WS101	TLS Lab Submissions	U1003	internal	AWS
WS103	Advising Intake Forms	U1004	restricted	GCP

`files.csv`

Each row is one file. `workspace_id` points to `workspaces.csv`, and `created_by_user_id` points to `users.csv`.

file_id	workspace_id	created_by_user_id	file_name	file_type	shared_externally
F001	WS101	U1001	tls_lab_reflection.docx	document	no
F006	WS103	U1003	weekly_intake_export.csv	spreadsheet	yes

Representation 1: One flat table

The simplest way to explain data to humans is often one wide spreadsheet. If we join the three CSV tables together, we can create a single flat report like this:

file_id	file_name	workspace_ name	cloud_ provider	data_ classification	owner_ name	creator_ name	shared_ externally
F001	tls_lab_ reflection.docx	TLS Lab Submissions	AWS	internal	Priya Shah	Ana Rivera	no
F004	practice_ questions.pdf	Midterm Review Notes	Azure	internal	Jordan Lee	Priya Shah	yes
F006	weekly_intake_ export.csv	Advising Intake Forms	GCP	restricted	Jordan Lee	Priya Shah	yes

Why a flat table is useful

- It is easy to read in a spreadsheet.
- It works well for quick filtering, sorting, and reporting.
- It is a good export format for a dashboard or one-time analysis.

What the flat table gives up

- The workspace name repeats for every file in that workspace.
- The owner name repeats for every file in that workspace.
- If the owner changes or the classification changes, many rows must be updated.
- Repeated sensitive values create more copies to protect.

A flat table is good for reporting, but it is usually not the best long-term storage model for constantly changing data.

Representation 2: Relational tables

The three CSV files in this week's folder represent a relational design. Instead of copying every fact into every row, the data is split into separate tables and connected with identifiers.

Main keys in this design

- Primary keys:
  - `users.user_id`
  - `workspaces.workspace_id`
  - `files.file_id`
- Foreign keys:
  - `workspaces.owner_user_id -> users.user_id`
  - `files.workspace_id -> workspaces.workspace_id`
  - `files.created_by_user_id -> users.user_id`

Why relational tables are useful

- Each fact usually has one main home.
- Updates happen in one place instead of many places.
- Duplicated data is reduced.
- Constraints can be enforced more clearly.
- This structure works well when accuracy and consistency matter.

What relational tables give up

- To answer a question, we often need joins.
- Queries can be harder for beginners to read.
- The model feels less convenient when an application usually wants one nested object all at once.

Example question

If we want a list of files with workspace and owner information, a relational database would combine the tables at query time:

```sql
SELECT
  f.file_name,
  w.workspace_name,
  owner.full_name AS owner_name,
  creator.full_name AS creator_name,
  w.cloud_provider
FROM files AS f
JOIN workspaces AS w
  ON f.workspace_id = w.workspace_id
```

```
JOIN users AS owner
  ON w.owner_user_id = owner.user_id
JOIN users AS creator
  ON f.created_by_user_id = creator.user_id;
```

This design is common in relational cloud databases such as PostgreSQL or MySQL services.

JSON primer: why it appears in cloud databases

Before moving into MongoDB-style documents, it helps to understand **JSON**. JSON is one of the most common ways modern software represents structured data.

What JSON is

JSON stands for **JavaScript Object Notation**. It is a text format for representing structured data in a way that is readable by both humans and machines.

A JSON document can represent: - one object - a list of objects - a mix of nested objects and lists

That makes JSON useful when data is not just one flat row. It can describe nested structure directly.

A short history of JSON

JSON grew out of JavaScript object literal syntax during the early web era. As web applications became more interactive in the late 1990s and early 2000s, developers needed a lightweight way to send structured data between browsers and servers.

JSON became popular because: - it was simpler and lighter than many older markup-based formats - it was easy for JavaScript-based web pages to parse - it was readable enough that people could inspect it directly

Even though JSON began with a strong connection to JavaScript, it is now used across many languages and systems. Python, Java, Go, Ruby, PHP, and many other languages can all read and write JSON.

Why JSON became so common

JSON became common because it is: - lightweight - text-based - easy to transmit over networks - easy for APIs to return - flexible enough for nested data

When a web application requests data from a server today, there is a good chance the response is JSON.

JSON syntax at a glance

JSON uses a small set of symbols: - curly braces `{ }` for an **object** - square brackets `[ ]` for an **array** - a colon `:` between a key and a value - commas `,` to separate items

Here is a simple JSON object:

```
{
  "user_id": "U1003",
  "full_name": "Priya Shah",
  "role": "teaching_assistant",
  "active": true
}
```

This object has: - **keys** such as `"user_id"` and `"role"` - **values** such as `"U1003"` and `"teaching_assistant"`

Here is a JSON array:

```
[
  "AWS",
  "Azure",
  "GCP"
]
```

Here is a nested example:

```
{
  "workspace_name": "TLS Lab Submissions",
  "owner": {
    "user_id": "U1003",
    "full_name": "Priya Shah"
  },
  "tags": ["class", "security", "week8"]
}
```

This is where JSON becomes especially useful. The value for `"owner"` is another object, and the value for `"tags"` is an array.

JSON value types

JSON has a small set of value types: - **string**: `"Jordan Lee"` - **number**: `42` - **object**: `{ "role": "student" }` - **array**: `["AWS", "GCP"]` - **boolean**: `true` or `false` - **null**: `null`

That limited set is part of why JSON is portable across systems.

Important JSON rules

JSON is flexible, but it is not free-form. Some important rules are:

1. **Keys must be in double quotes**
    - valid: `"user_id"`
    - not valid JSON: `'user_id'`
2. **Strings must use double quotes**
    - valid: `"Ana Rivera"`
    - not valid JSON: `'Ana Rivera'`
3. **Booleans are lowercase**
    - valid: `true`, `false`
    - not valid JSON: `True`, `False`
4. **No trailing commas**
    - valid:

```
{
  "a": 1,
  "b": 2
}
```

- not valid:

```
{
  "a": 1,
  "b": 2,
}
```

5. **Comments are not part of standard JSON**
    - this is not valid JSON:

```
{
  "user_id": "U1001"
  // student record
}
```

6. **Every opening brace or bracket must have a matching closing brace or bracket**

Because JSON is strict, tiny formatting mistakes can make data invalid.

JSON versus CSV

CSV and JSON are both text formats, but they solve different problems.

Format	Best for	Shape
CSV	flat tabular data	rows and columns

Format	Best for	Shape
JSON	nested structured data	objects and arrays

CSV is great when: - each row has the same columns - the structure is flat - the goal is spreadsheet-style viewing or quick export

JSON is great when: - data has nested structure - one object contains other objects - one record contains lists - an API needs to send structured application data

Common JSON use cases

JSON appears in many places: - web API requests and responses - front-end and back-end data exchange - configuration files - cloud service payloads - event logging and telemetry - document-oriented databases

For example: - a weather API may return current temperature, forecast details, and location fields in one JSON response - a web app may send a login request as JSON - a cloud service may return resource settings and policy details as JSON

Why JSON matters before MongoDB

MongoDB documents are often explained in a JSON-like format because JSON is easy for people to read. Internally, MongoDB uses a related binary format called **BSON**, but the developer-facing idea is very close to JSON objects and arrays.

That means MongoDB feels familiar to many developers because: - web apps already use JSON heavily - nested JSON maps naturally to nested documents - one JSON-like document can match one application object

So before learning MongoDB, students should be comfortable reading and thinking in JSON.

Representation 3: MongoDB-style documents

The same facts can also be stored as documents rather than rows spread across several tables. For a MongoDB-style design, one option is to store each workspace as a single document and embed the files inside it.

```
{
  "_id": "WS101",
  "workspace_name": "TLS Lab Submissions",
  "data_classification": "internal",
  "cloud_provider": "AWS",
  "owner": {
    "user_id": "U1003",
    "full_name": "Priya Shah",
    "role": "teaching_assistant"
```

```
  },
  "files": [
    {
      "file_id": "F001",
      "created_by_user_id": "U1001",
      "creator_name": "Ana Rivera",
      "file_name": "tls_lab_reflection.docx",
      "file_type": "document",
      "shared_externally": false
    },
    {
      "file_id": "F002",
      "created_by_user_id": "U1002",
      "creator_name": "Malik Thompson",
      "file_name": "certificate_notes.csv",
      "file_type": "spreadsheet",
      "shared_externally": false
    }
  ]
}
```

Why a document model is useful

- One read can return a whole workspace with its files.
- The shape matches JSON used in many web APIs.
- It is often intuitive for cloud applications that think in nested objects.
- It can reduce application-side join logic for read-heavy workloads.

What the document model gives up

- Some values may be duplicated across documents.
- Updating repeated information can become harder.
- Large embedded arrays may become awkward if they grow quickly.
- Some relationships are better handled with references instead of embedding.

MongoDB does not force one single pattern. Teams choose between embedding and referencing based on how the application actually reads and writes data.

When each representation makes sense

Representation	Best fit	Main strength	Main risk
Flat table	One-time reports, exports, spreadsheets, quick auditing views	Easy for humans to read	Heavy duplication and update pain

Representation	Best fit	Main strength	Main risk
Relational tables	Systems with frequent updates, strong consistency needs, clear relationships	Less redundancy and stronger data integrity	Joins add complexity
Document model	Cloud apps that usually fetch one nested object at a time	Natural fit for JSON and app-driven reads	Duplication and document-shape tradeoffs

The important point is that the same facts can be validly represented in more than one way. The correct answer depends on the use case, not on a rule that one model is always superior.

Why this matters in the cloud

Cloud databases add convenience, but they do not remove design decisions. A managed service can help with hosting, backups, scaling, and availability. It does not decide: - which fields you duplicate - who can access sensitive records - how long data should be retained - whether your model supports least privilege and data minimization

For example: - a flat export is convenient, but it may spread restricted values into more files than necessary - a relational design may simplify governance because one field has one authoritative location - a document model may improve application performance, but embedded personal data can widen the blast radius if documents are overshared

Security, privacy, and ethics connection

Data models are not just technical choices. They also affect how safely and responsibly information is handled.

Questions to ask: - Are we copying sensitive fields into too many places? - Can we separate restricted data from general-use data? - Does this model make least privilege easier or harder? - If the system is cloud-hosted, who is responsible for region choice, access control, and retention?

In this course, the goal is not to memorize a “best” model. The goal is to explain why one model is more defensible for a given task.

Takeaways for class

- A table is a structured collection of records, not just a spreadsheet tab.
- Multiple tables can represent one system more accurately than one flat sheet.
- JSON is a common text format for nested structured data.
- MongoDB documents can represent the same facts in a nested, application-friendly way.
- Use cases should drive the model choice.

- Security and privacy obligations remain, no matter where the database is hosted.